SQL PROCEDURAL

Dans les différentes parties de ce chapitre, on utilisera la base de données suivante :

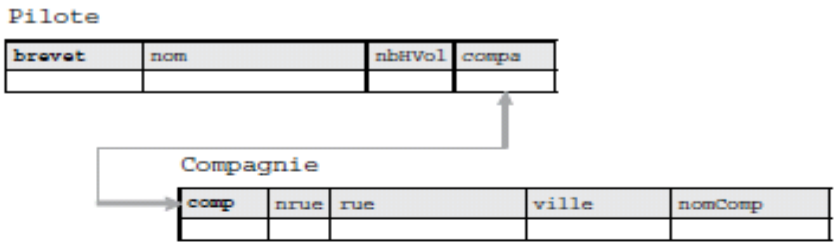


Tableau 1-3 Contraintes en ligne et nommées

Tables	Contraintes

`CREATE TABLE Compagnie`
`(comp CHAR(4), nrue INTEGER(3),`
`rue CHAR(20), ville CHAR(15) DEFAULT 'Paris'`
`COMMENT 'Par defaut : Paris',`
`nomComp CHAR(15) NOT NULL,`
`CONSTRAINT pk_Compagnie PRIMARY KEY(comp));`
	Deux contraintes en ligne et une contrainte nommée de clé primaire.

`CREATE TABLE Pilote`
`(brevet CHAR(6), nom CHAR(15) NOT NULL,`
`nbHVol DECIMAL(7,2), compa CHAR(4),`
`CONSTRAINT pk_Pilote PRIMARY KEY(brevet),`
`CONSTRAINT ck_nbHVol`
`CHECK(nbHVol BETWEEN 0 AND 20000),`
`CONSTRAINT un_nom UNIQUE (nom),`
`CONSTRAINT fk_Pil_compa_Comp`
`FOREIGN KEY (compa)`
`REFERENCES Compagnie(comp));`
	Une contrainte en ligne et quatre contraintes nommées :
	• Clé primaire
	• `NOT NULL`
	• `CHECK` (nombre d'heures de vol compris entre 0 et 20 000)
	• `UNIQUE` (homonymes interdits)
	• Clé étrangère

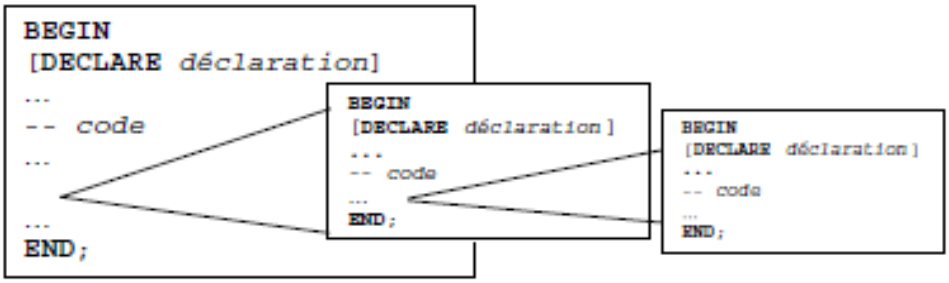
Partie 1 : Bases du langage de programmation

1. Introduction

- La norme SQL n'intègre pas Les structures de contrôle habituelles d'un langage (IF, WHILE…)
- Elles apparaissent dans une sous-partie optionnelle de la norme (ISO/IEC 9075-5:1996. *Flow-control statements*).
- Le langage procédural de MySQL est une extension de SQL, permettant de faire cohabiter les structures de contrôle avec des instructions SQL.

2. Généralités

2.1. Structure d'un bloc d'instruction



- Pour exécuter un bloc il faut l'inclure dans une procédure cataloguée.

2.2. Portée des objets

- Un objet (variable, curseur ou exception) déclaré dans un bloc est accessible dans les sous-blocs.
- Un objet déclaré dans un sous-bloc n'est pas visible du bloc supérieur.

2.3. Casse et lisibilité

- Mysql procédural est non « *case sensitive* ». numBrevet et NumBREVET désignent le même identificateur
- Lisibilité du code :

Peu lisible	C'est mieux

`IF x>y THEN SET max:=x;ELSE SET max:=y;END IF;`	`IF x>y THEN`
	`  SET max:=x;`
	`ELSE`
	`  SET max:=y;`
	`END IF;`

2.4. Identificateurs

- Permet de nommer un objet utilisé dans un bloc
- Commence par une lettre (ou un chiffre)
- N'est pas limité en nombre de caractères

Exemple :

Autorisés	Interdits
`t2`	`moi&toi` (symbole « & »)
`code_brevet`	`debit-credit` (symbole « - »)
`2nombresMysql`	`on/off` (symbole « / »)
`_t`	`code brevet` (symbole espace)

2.5. Commentaires

- Monolignes : commençant avec le symbole « -- » et finissant à la fin de la ligne
- multilignes, commençant par « `/*` » et finissant par « `*/` »

3. Variables

- Les variables qui sont déclarées (et éventuellement initialisées) par la directive DECLARE
- Deux types de variables sont disponibles sous MySQL :
  - scalaires : recevant une seule valeur d'un type SQL (ex : colonne d'une table)
  - externes : définies dans la session

3.1. Variables scalaires

- Syntaxe :

```
DECLARE nomVariable1[,nomVariable2...] typeMSQL [DEFAULT expression];
```

où **DEFAULT** permet d'initialiser la (ou les) variable(s).

Le tableau suivant décrit quelques exemples :

Déclarations	Commentaires
`DECLARE v_dateNaissance DATE;`	Déclare la variable sans l'initialiser. Équivalent à `SET v_dateNaissance := NULL;`
`DECLARE v_capacite SMALLINT(4) DEFAULT 999;`	Initialise la variable à 999.
`DECLARE v_trouve BOOLEAN DEFAULT TRUE;`	Initialise la variable à vrai (1).
`DECLARE v_Dans2jours DATE DEFAULT ADDDATE(SYSDATE(),2);`	Initialise la variable à dans 2 jours.

3.2. Affectations

Il existe plusieurs possibilités pour affecter une valeur à une variable :

- l'affectation comme on la connaît dans les langages de programmation : SET *variable* := *expression*
- la directive DEFAULT (voir plus haut)
- la directive INTO d'une requête (SELECT… INTO *variable* FROM…)

3.3. Restrictions

Le type tableau (*array*) n'est pas encore présent dans le langage de MySQL. Cela peut être pénalisant quand on désire travailler en interne avec des résultats d'extractions de taille moyenne.

3.4. Résolution de noms

- Dans des instructions SQL, le nom de la variable est prioritairement interprété au détriment de la colonne de la table (de même nom)

Exemple : Ici , l'instruction DELETE supprime tous les pilotes de la table (et non pas seulement le pilote de nom 'Bougara'), car MySQL considère les deux identificateurs comme étant la même variable

```
DECLARE   nom   VARCHAR(16)   DEFAULT 'Bougara';
DELETE    FROM  Pilote  WHERE  nom = nom ;
```

- Solutions:
  - nommer toutes les variables différemment des colonnes (utiliser un préfixe, par exemple).
  - utiliser une étiquette de bloc

Exemple :

Préfixer les variables	Étiquette de bloc (préfixe pas opérationnel)

`DECLARE v_nom VARCHAR(16)`
`  DEFAULT 'Placide Fresnais';`
`…`
`DELETE FROM Pilote WHERE nom = v_nom;`
`--ou`
`DELETE FROM Pilote WHERE v_nom = nom;`
	`principal: BEGIN`
	`DECLARE nom VARCHAR(16)`
	`  DEFAULT 'Placide Fresnais';`
	`DELETE  FROM Pilote`
	`  WHERE principal.nom = nom;`
	`END principal;`

3.5. Opérateurs

- Les opérateurs SQL étudiés au chapitre 'Rappel SQL' (logiques, arithmétiques, de concaténation…) sont disponibles au sein d'un sous-programme.
- Les règles de priorité sont les mêmes que dans le cas de SQL.
- L'opérateur IS NULL permet de tester une formule avec la valeur NULL. Toute expression arithmétique contenant une valeur nulle est évaluée à NULL.

Exemple :

Code MySQL	Commentaires

`DECLARE v_compteur INT(3) DEFAULT 0;`
`DECLARE v_boolean BOOLEAN;`
`DECLARE v_nombre INT(3);`
	Trois déclarations dont une avec initialisation.

`SET v_compteur := v_compteur+1;`	Incrémentation de `v_compteur` (opérateur +)
`SET v_boolean := (v_compteur=v_nombre);`	`v_boolean` reçoit **NULL**, car la condition est fausse.
`SET v_boolean := (v_nombre IS NULL);`	`v_boolean` reçoit **TRUE** (1 en fait), car la condition est vraie.

3.6. Variables de session

- Il est possible de passer en paramètres d'entrée d'un bloc des variables externes. Ces variables sont dites de session (*user variables*).
- Elles n'existent que durant la session.
- On déclare ces variables en ligne de commande à l'aide du symbole « @ ».

```
SET @var1 = expression1  [, @var2 = expression2] ...
```

Exemple :

Code MySQL	Résultat

`SET @vs_num = 'PL-4'$`
`SET @vs_hvol = 15$`
`…`
`BEGIN`
`  DECLARE v_nom  CHAR(16);`
`  DECLARE v_nbHVol DECIMAL(7,2);`
`  SELECT nom,nbHVol`
`  INTO v_nom, v_nbHVol FROM Pilote`
`  WHERE brevet = @vs_num;`
`  SET v_nbHVol := v_nbHVol + @vs_hvol;`
`  SELECT v_nom, v_nbHVol;`
`END;`
	`+----------+----------+`
	`| v_nom  | v_nbHVol |`
	`+----------+----------+`
	`| Placide Fresnais |  2465.00 |`
	`+----------+----------+`

3.7. Conventions recommandées

- Intérêt : lisibles et maintenables

Objet	Convention	Exemple
Variable	v_nomVariable	v_compteur
Constante	c_nomConstante	c_pi
Variable de session (globale)	vs_nomVariable	vs_brevet

3.8. Exemple de Bloc

- Restriction Mysql : pas possible d'exécuter des blocs anonymes (sous-programme sans nom et qui n'est pas stocké dans la base)
- Un bloc doit être inclus dans une procédure cataloguée a appellée dans l'interface de commande

Exemple : extraction de nombre d'heures de vol du pilote de nom 'Placide Fresnais'.

La redéfinition du délimiteur à « $ » permet d'utiliser, dans le bloc, le symbole « ; » pour terminer chaque instruction.

Préfixer les variables	Commentaire
`delimiter $` `SET @vs_nom = 'Placide Fresnais'$`	Déclaration du délimiteur et d'une variable de session.
`DROP PROCEDURE sp1$`	Suppression de la procédure.
`CREATE PROCEDURE sp1()`	Création de la procédure.

`BEGIN`
`  DECLARE v_nbHVol DECIMAL(7,2);`
`  SELECT nbHVol INTO v_nbHVol`
`  FROM Pilote WHERE nom = @vs_nom;`
	Bloc d'instructions.

`  SELECT v_nbHVol;`	Trace du résultat.
`END;` `$`	Fin du bloc
`CALL sp1()$`	Appel de la procédure.

3.9. Structures de contrôle

3.9.1. Structures conditionnelles

Deux structures pour programmer des actions conditionnées : la structure IF et la structure CASE

3.9.1.1. Trois formes de IF

IF-THEN	IF-THEN-ELSE	IF-THEN-ELSEIF

`IF condition THEN`
`  instructions;`
`END IF;`
	`IF condition THEN`
	`  instructions;`
	`ELSE`
	`  instructions;`
	`END IF;`
		`IF condition1 THEN`
		`  instructions;`
		`ELSEIF condition2 THEN`
		`  instructions2;`
		`  ELSE`
		`  instructions3;`
		`END IF;`

Exemple :

```
BEGIN
  DECLARE v_telephone CHAR(14)
          DEFAULT '06-76-85-14-89';
  IF SUBSTR(v_telephone,1,2)='06' THEN
     SELECT "C'est un portable";
  ELSE
     SELECT "C'est un fixe...";
  END IF;
END;
```

Pas de structure FOR pour l'instant.

3.9.1.2. Structure CASE

Deux façons d'utiliser le CASE :

CASE	*searched* CASE

CASE *variable*
  WHEN *expr1* **THEN** *instructions1;*
  WHEN *expr2* **THEN** *instructions2;*
  …
  WHEN *exprN* **THEN** *instructionsN;*
  [**ELSE** *instructionsN+1;*]
END CASE;
	CASE
	  WHEN *condition1* **THEN** *instructions1;*
	  WHEN *condition2* **THEN** *instructions2;*
	  …
	  WHEN *conditionN* **THEN** *instructionsN;*
	  [**ELSE** *instructionsN+1;*]
	END CASE;

Exemple : de cas où le CASE est plus adapté que le IF

IF	CASE
	BEGIN DECLARE v_mention CHAR(2); DECLARE v_note DECIMAL(4,2) DEFAULT 9.8; ...

IF v_note >= 16 THEN
  SET v_mention := 'TB';
  ELSEIF v_note >= 14 THEN
  SET v_mention := 'B';
  ELSEIF v_note >= 12 THEN
  SET v_mention := 'AB';
  ELSEIF v_note >= 10 THEN
  SET v_mention := 'P';
  ELSE
  SET v_mention := 'R';
END IF;
...
	CASE
	  WHEN v_note >= 16 **THEN** **SET** v_mention := 'TB';
	  WHEN v_note >= 14 **THEN** **SET** v_mention := 'B';
	  WHEN v_note >= 12 **THEN** **SET** v_mention := 'AB';
	  WHEN v_note >= 10 **THEN** **SET** v_mention := 'P';
	  ELSE **SET** v_mention := 'R';
	END CASE;
	...

3.9.2. Structures de contrôle répétitives

Mysql intègre les structures de contrôles WHILE, REPEAT et LOOP mais pas encore le FOR.

3.9.2.1. Structure WHILE

Syntaxe :

```
[etiquette:] WHILE condition DO
   instructions;
END WHILE [etiquette];
```

Exemple 1 : calcule la somme des 100 premiers entiers.

Condition simple

```
DECLARE v_somme INT DEFAULT 0;
DECLARE v_entier SMALLINT DEFAULT 1;

WHILE (v_entier <= 100) DO
 SET v_somme  := v_somme+v_entier;
 SET v_entier := v_entier+1;
END WHILE;
SELECT v_somme;
```

```
+---------+
| v_somme |
+---------+
|    5050 |
+---------+
```

Exemple 2 : recherche le premier numéro 4 dans une chaîne de caractères

Condition composée

```
DECLARE v_telephone CHAR(14)
       DEFAULT '06-76-85-14-89';
DECLARE v_trouve BOOLEAN
        DEFAULT  FALSE;
DECLARE v_indice SMALLINT DEFAULT  1;
WHILE (v_indice <= 14 AND NOT v_trouve) DO
  IF SUBSTR(v_telephone,v_indice,1) = '4'
   THEN
      SET v_trouve := TRUE;
   ELSE
      SET v_indice := v_indice + 1;
  END IF;
END WHILE;
IF v_trouve THEN
   SELECT CONCAT('Trouvé 4 à l''indice :
',v_indice);
END IF;
```

Trouvé 4 à l'indice : 11

3.9.2.2. Structure « REPEAT .. UNTIL »

Syntaxe :

```
[etiquette:] REPEAT
    instructions;
UNTIL condition END REPEAT [etiquette];
```

Exemples : Reprenons les 2 exemples du WHILE

Condition simple

```
REPEAT
  SET v_somme  := v_somme + v_entier;
  SET v_entier := v_entier + 1;
UNTIL v_entier > 100 END REPEAT;
```

Condition composée

```
REPEAT
  IF SUBSTR(v_telephone,v_indice,1) = '4'
    THEN
     SET v_trouve := TRUE;
    ELSE
      SET v_indice := v_indice + 1;
  END IF;
UNTIL (v_indice > 14 OR v_trouve)
    END REPEAT;
IF v_trouve THEN
  SELECT CONCAT('Trouvé 4 à l''indice :
',v_indice);
END IF;
```

3.9.2.3. Structure boucle sans fin « LOOP »

Syntaxe :

```
[etiquette:] LOOP
    instructions;
END LOOP [etiquette];
```

Exemple : Reprenons l'exemple qui calcul la somme des 100 premiers entiers en utilisant deux boucles sans fin. La directive ITERATE force à reprendre l'exécution au début de la boucle.

Avec LEAVE

```
bouclel: LOOP
  SET v_somme  := v_somme + v_entier;
  SET v_entier := v_entier + 1;
  IF v_entier > 100 THEN
     LEAVE bouclel;
  END IF;
END LOOP bouclel;
```

Avec ITERATE

```
bouclel: LOOP
  SET v_somme  := v_somme + v_entier;
  SET v_entier := v_entier + 1;
  IF v_entier <= 100 THEN
     ITERATE bouclel;
  END IF;
  LEAVE bouclel;
END LOOP bouclel;
```

Notes :

- LEAVE peut être aussi utilisé pour sortir d'un bloc (s'il est étiqueté).
- LEAVE et ITERATE peuvent aussi être employés au sein de structures REPEAT ou WHILE.
- LOOP devient sans fin si vous n'utilisez pas l'instruction LEAVE qui passe en séquence du END LOOP.

3.10. Interactions avec la base

On décrit ici les mécanismes que MySQL offre pour interfacer un sous-programme avec une base de données.

3.10.1. Extraire des données

Syntaxe :

```
SELECT col1 [,col2 ...]INTO variable1 [,variable2 ...]
       FROM nomTable ...;
```

Remarque : la directive INTO permet de charger des variables à partir de valeurs de colonnes.

Exemple 1 : extraction de la colonne « compa » pour le pilote de code 'PL-2' dans différents contextes :

Code MYSQL	Commentaires

`BEGIN`
`  DECLARE v_comp VARCHAR(15);`
`  SELECT compa INTO v_comp`
`  FROM Pilote WHERE brevet='PL-2';`
`  ...`
`END;`
	Chargement d'une variable locale à un bloc. Nécessité d'appeler par la suite cette procédure (CALL).

`SET @vs_compa=''$`
`SELECT compa INTO @vs_compa FROM Pilote`
`WHERE brevet='PL-2'$`
`...`
	Chargement d'une variable de session hors d'un sous-programme.

`SET @vs_compa=''$`
`CREATE PROCEDURE sp1()`
`BEGIN`
`  SELECT compa INTO @vs_compa`
`  FROM Pilote WHERE brevet='PL-2';`
`  ...`
`END;`
	Chargement d'une variable de session dans un sous-programme.

Note : Pour traiter des requêtes renvoyant plusieurs enregistrements, il faudra utiliser des curseurs (étudiés plus loin).

Exemple 2 (utilisation de fonction monoligne):
Chargement de la variable avec le nom du pilote de code 'PL-1' en majuscules.

Monoligne

`BEGIN`
`  DECLARE v_nomEnMAJUSCULES CHAR(20);`
`  SELECT UPPER(nom)`
`  INTO v_nomEnMAJUSCULES`
`  FROM Pilote WHERE brevet = 'PL-1';`
`SELECT v_nomEnMAJUSCULES;`
`END;`

`+-------------------+`
`| v_nomEnMAJUSCULES |`
`+-------------------+`
`| GRATIEN VIEL  |`
`+-------------------+`

Exemple 3 (utilisation de fonction multiligne) :
Affectation à la variable le maximum du nombre d'heures de vol, tous pilotes confondus.

Multiligne

```
BEGIN
  DECLARE v_plusGrandHVol DECIMAL(7,2);
  SELECT MAX(nbHVol) INTO v_plusGrandHVol
         FROM Pilote;
SELECT v_plusGrandHVol ;
END;
```

```
+-----------------+
| v_plusGrandHVol |
+-----------------+
|         2450.00 |
+-----------------+
```

3.10.2.Manipuler des données

Les instructions de manipulation, par un sous-programme de données sont les mêmes que celles utilisées par SQL, à savoir INSERT, UPDATE et DELETE.

3.10.2.1. Insertions

Exemple :

Code MySQL	Commentaires

`BEGIN`
`DECLARE v_brevet VARCHAR(6) DEFAULT 'PL-7';`
`DECLARE v_nom VARCHAR(6);`
`DECLARE v_HVol DECIMAL(7,2) DEFAULT 0;`
`DECLARE v_comp VARCHAR(6);`
	Déclaration des variables locales au bloc.

`INSERT INTO Pilote VALUES` `('PL-6', 'Jules Ente', 3000, 'AF');`	Insertion d'un enregistrement en renseignant les colonnes par des constantes.

`SET v_nom := 'Fabrice Peyrard';`
`SET v_comp := 'SING';`
`INSERT INTO Pilote VALUES`
`(v_brevet,v_nom,v_HVol,v_comp);`
`END;`
	Insertion d'un enregistrement en renseignant les colonnes par des variables locales.

Note : Dans le cas d'une erreur, une exception qui précise la nature du problème est levée et peut être interceptée par la directive HANDLER (voir plus loin). Si une telle directive n'existe pas dans le bloc qui contient l'instruction INSERT, la première exception fera s'interrompre le programme.

3.10.2.2. Modifications

Syntaxe :

```
UPDATE nomTable
  SET col1 = { variable1 | expression1 | autrecol | (requête) }
    [,col2 = ...]
  [WHERE ... ];
```

Exemple : modification de différents enregistrements

Code MySQL	Commentaires
`BEGIN` `DECLARE v_dureeVol DECIMAL(3,1) DEFAULT 4.8;`	Déclaration.

`UPDATE Pilote`
`SET nbHVol= nbHVol + v_dureeVol`
`WHERE brevet= 'PL-6';`
	Modification d'un enregistrement de la table `Pilote` en utilisant une variable.

`UPDATE Pilote`
`SET nbHVol= nbHVol + 10`
`WHERE compa = 'AF';`
`END;`
	Modification de plusieurs enregistrements de la table `Pilote` en utilisant une constante.

3.10.2.3. Suppressions

Syntaxe :

```
DELETE FROM nomTable
  [WHERE col1 = { variable1 | expression1 | autrecol | (requête) }
    [,col2 = ...] ];
```

Exemple : Suppression de différents enregistrements

Code MYSQL

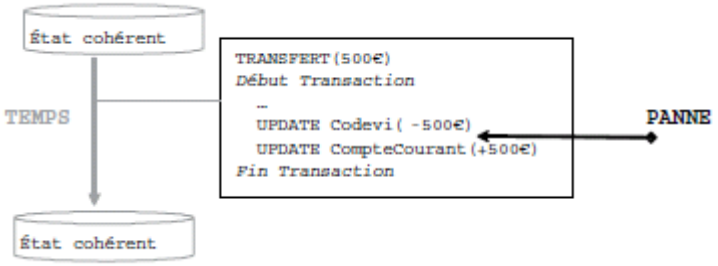
```
BEGIN
 DECLARE v_hVolMini DECIMAL(7,2) DEFAULT 1000.00;

 DELETE FROM Pilote WHERE nbHVol < v_hVolMini;
  DELETE FROM Pilote WHERE brevet = 'PL-3';
```

3.11. Transactions

Définition : Une transaction est un bloc d'instructions LMD faisant passer la base de données d'un état cohérent à un autre état cohérent. Toutes les instructions de la transaction doivent s'exécuter entièrement ou pas du tout. Si une erreur survient au cours d'une transaction, toutes les instructions déjà exécutées sont annulées.

Exemple : transfert d'un compte 1(Codevi) vers un compte 2(CompteCourant).



Propriétés : (ACID)

Une transaction assure :

- l'Atomicité des instructions qui sont considérées comme une seule opération (principe du tout ou rien)
- la Cohérence : passage d'un état cohérent de la base à un autre état cohérent
- l'Isolation des transactions entre elles
- la Durabilité des opérations : les mises à jour perdurent même si une panne se produit après la transaction

3.11.1. Début et fin d'une transaction

Deux instructions permettent de marquer le début d'une transaction :

1. START TRANSACTION

ou

2. BEGIN

Note :

1. Entre BEGIN et END d'un programme MySQL, il est possible d'écrire plusieurs transactions.
2. Le fait de commencer une transaction termine implicitement celle qui précédait.

Une transaction se termine :

a) explicitement par les instructions :

1. SQL COMMIT

ou

2. ROLLBACK

b) implicitement :

1. à la première commande SQL du LDD rencontrée (CREATE, ALTER, DROP…)
2. à la fin normale d'une session utilisateur avec déconnexion
3. à la fin anormale d'une session utilisateur (sans déconnexion).

3.11.2. Mode de validation

- Deux modes de fonctionnement sont possibles :
  - celui par défaut (autocommit) qui valide systématiquement toutes les instructions reçues par la base. Dans ce mode, il est impossible de revenir en arrière afin d'annuler une instruction.
  - Le mode inverse (autocommit off) qui se déclare à l'aide de l'instruction suivante

Syntaxe :

```
SET AUTOCOMMIT = {0 | 1}
```

- Le tableau suivant précise la validité de la transaction en fonction des événements possibles :

Événement	Validité
COMMIT	Transaction validée.
ROLLBACK Commande SQL (LDD ou LCD). Fin anormale d'une session.	Transaction non validée.

3.11.3. Exemple de transaction

Etant donnée la procédure suivante :

```
delimiter $
DROP PROCEDURE sp1$
CREATE PROCEDURE sp1()
BEGIN
 SET AUTOCOMMIT = 0;
 INSERT INTO TableaVous VALUES (...);
END;
$
--appel de la transaction
CALL sp1()$
SELECT * FROM TableaVous$
```

Etape 1 :

Exécution du bloc dans l'interface,

Etape 2 :

Déconnection soit en cassant la fenêtre (icône en haut à droite), soit proprement avec exit.

Etape 3 :

- On se reconnecte,
- Résultat : l'enregistrement n'est pas présent dans la table 'TableaVous'. Même quand la fin du programme est normale, la transaction n'est pas validée (car il manque COMMIT).

Etape 4 :

- Relance du bloc en ajoutant l'instruction COMMIT après l'insertion.
- Résultat : l'enregistrement est présent dans la table, même après une déconnexion douce ou dure.

3.11.4. Contrôle des transactions

Il est possible de découper une transaction en insérant des points de validation (**savepoints**) qui rendent possible l'annulation de tout ou partie des opérations composant la dite transaction.

L'instruction SAVEPOINT déclare un point de validation :
Syntaxe : **SAVEPOINT** Label ;

Exemple : Le tableau suivant décrit une transaction MySQL découpée en trois parties. Le programmeur aura le choix entre les instructions `ROLLBACK TO SAVEPOINT` indiquées en commentaire pour valider tout ou partie de la transaction. Il faudra finalement choisir entre `COMMIT` et `ROLLBACK`.

Code MYSQL	Commentaires

`BEGIN`
`SET AUTOCOMMIT = 0;`
`INSERT INTO Compagnie VALUES`
`('C2',2, 'Place Brassens', 'Blagnac', 'Easy Jet');`
	Première partie de la transaction.

`SAVEPOINT P1;`
`UPDATE Compagnie SET nrue = 125`
`WHERE comp = 'AF';`
`UPDATE Compagnie SET ville = 'Castanet'`
`WHERE comp = 'C1';`
	Deuxième partie de la transaction.

`SAVEPOINT P2;` `DELETE FROM Compagnie WHERE comp = 'C1';`	Troisième partie de la transaction.
`-- ROLLBACK TO SAVEPOINT P1;`	Première partie à valider.
`-- ROLLBACK TO SAVEPOINT P2;`	Deuxième partie à valider.
`ROLLBACK;`	Tout à invalider.
`COMMIT;` `END;`	Valide la ou les sous-parties.

Note : Il n'est pas possible d'invalider par ROLLBACK une commande SQL du LDD rencontrée (CREATE, ALTER, DROP…).

Partie 2 : Programmation avancée

Cette partie est consacrée à des caractéristiques avancées du langage procédural de MySQL *Stored*

1. Procédures stockées : (stored procedures ou stored routines)

Ce sont des fonctions ou procédures « cataloguées » (ou « stockées »)

1.1. Généralités

Une procédure peut être appelée à l'aide :

- de l'interface de commande (par CALL),
- dans un programme externe (Java, PHP, C…),
- par d'autres procédures ou fonctions,
- dans le corps d'un déclencheur.

Les fonctions peuvent être invoquées

- dans une instruction SQL (SELECT, INSERT, et UPDATE),
- dans une expression (affectation de variable ou calcul).

1.2. Les principaux avantages (stockées côté serveur) :

- La modularité : un sous-programme peut être composé d'autres blocs d'instructions.
- Réutilisabilité : Un sous-programme peut aussi être réutilisable, car il peut être appelé par un autre.
- La portabilité : un sous-programme est indépendant du système d'exploitation qui héberge le serveur MySQL. En changeant de système, les applicatifs n'ont pas à être modifiés.
- L'intégration avec les données des tables : on retrouvera avec ce langage procédural tous les types de données et d'instructions disponibles sous MySQL, des mécanismes pour parcourir des résultats de requêtes (curseurs), pour traiter des erreurs (handlers) et pour programmer des transactions (COMMIT, ROLLBACK, SAVEPOINT).
- La sécurité : car les sous-programmes s'exécutent dans un environnement a priori sécurisé (SGBD) où il est plus facile de garder la maîtrise sur les ordres SQL exécutés et donc sur les agissements des utilisateurs.

1.3. Syntaxe de création de procédure et fonction

```
CREATE { PROCEDURE | FUNCTION }
       nomSousProgramme [(...) ] [RETURNS typeMSQL ]
BEGIN
[DECLARE déclaration ]; ...
instructions MySQL;
      BEGIN
      [DECLARE déclaration ];...
      ...
      ...
      instructions MySQL;
      END;
…
END;
délimiteur
```

où :

Délimiteur : délimiteur de commandes différent de « ; » (symbole utilisé obligatoirement en fin de chaque déclaration et instruction du langage procédural de MySQL

1.4. Compilation et appel de sous-programme

1.4.1. Compilation :

Pour compiler ces sous-programmes à partir de l'interface de commande, il faut procéder comme suit :

```
delimiter $
Sous programme ;
```

```
$
```

1.4.2. Appel

La procédure s'appelle toujours par CALL, la fonction par son nom.

Exemple :

```
a)
        delimiter ;
        SET @vs_compa = 'AF';
        SET @vs_nompil = '';
        SET @vs_heures = '';
        CALL PlusExperimente (@vs_compa, @vs_nompil, @vs_heures);
b)
        delimiter ;
        SELECT EffectifsHeure ('AF',300) ;
c)
        SET @vs_compa = NULL$
        SET @vs_nompil = ''$
        SET @vs_heures = ''$
        CREATE PROCEDURE test.sp1()
        BEGIN
                CALL PlusExperimente (@vs_compa,@vs_nompil,@vs_heures);
        END;
        $


        CALL test.sp1()$
```

1.5. Destruction d'un sous-programme

Pour supprimer un sous-programme, si vous n'êtes pas son créateur, le privilège ALTER ROUTINE est requis sur la base de données.

Syntaxe : `DROP {PROCEDURE | FUNCTION} [IF EXISTS] [nomBase.]nomSousProg`

2. Programmation des curseurs

Un curseur est une zone mémoire qui est générée côté serveur (mise en cache) et qui permet de traiter individuellement chaque ligne renvoyée par un SELECT.

- Le curseur est décrit après les variables et avant les exceptions
- Il est ouvert dans le code du programme et va ensuite se charger en extrayant les données de la base
- Le programme parcour tout le curseur en récupérant les lignes une par une dans des variables locales
- Le curseur est ensuite fermé.

2.1. Instructions

Les instructions disponibles pour travailler avec des curseurs sont définies dans le tableau suivant :

1. *Déclaration de curseur : Cursor For*

   Exemple :
   ```
   DECLARE curs1 CURSOR FOR
                 SELECT brevet,nbHVol,comp
                 FROM   Pilote
                 WHERE  comp = 'AF';
   ```
2. *Ouverture de curseur (chargement des lignes) : Open*

   Exemple : `OPEN curs1;`

   Note : Aucune exception n'est levée si la requête ne ramène aucune ligne.

3. *Recherche d'un ligne du curseur : Fetch*

   Exemple: `FETCH curs1 INTO var1, var2, var3;`

   Note : Positionnement sur la ligne suivante et chargement de l'enregistrement courant dans une ou plusieurs variables.
4. *Fermeture de curseur : Close*

   Exemple : `CLOSE curs1;`

3. Gestion des exceptions

Afin d'éviter qu'un programme ne s'arrête dès la première erreur suite à une instruction SQL, il est indispensable de prévoir les cas potentiels d'erreurs et d'associer à chacun de ces cas la programmation d'une exception (handler dans le vocabulaire de MySQL).

- Une exception MySQL correspond à une condition d'erreur
- Les exceptions peuvent se paramétrer dans un sous-programme (fonction ou procédure cataloguée) ou un déclencheur
- Une exception est détectée (ou « levée ») si elle est prévue dans un handler au cours de l'exécution d'un bloc (entre BEGIN et END)
- Une fois levée, elle fait continuer ou sortir du bloc le programme après avoir réalisé une ou plusieurs instructions que le programmeur aura explicitement spécifiées.

3.1. Syntaxe

```
DECLARE       { CONTINUE | EXIT }
      HANDLER FOR
             { SQLSTATE [VALUE] 'valeur_sqlstate' | nomException | SQLWARNING
             | NOT FOUND | SQLEXCEPTION | code_erreur_mysql }
      instructions_MySQL;
```

où :

- CONTINUE est une directive (appelée handler) qui force la poursuite de l'exécution de programme lorsqu'il se passe un événement prévu dans la clause FOR.
- EXIT est une directive qui fait sortir l'exécution du bloc courant (entre BEGIN et END).

Et Les événements sont :

- SQLSTATE permet de couvrir toutes les erreurs d'un état donné.
- nomException s'applique à la gestion des exceptions nommées (étudiées plus loin).
- SQLWARNING permet de couvrir toutes les erreurs d'état SQLSTATE débutant par 01.
- NOT FOUND permet de couvrir toutes les erreurs d'état SQLSTATE débutant par 02.
- SQLEXCEPTION gère toutes les erreurs qui ne sont ni gérées par SQLWARNING ni par NOT FOUND.
- code_erreur_mysql désigne un code erreur Mysql

instructions_MySQL : une ou plusieurs instructions du langage de MySQL (bloc, appel possibles par CALL d'une fonction ou d'une procédure stockée).

Exemple :

```
CREATE PROCEDURE bdsoutou.procException1
       (IN p_comp VARCHAR(4))
BEGIN
 DECLARE flagPlusDun  BOOLEAN DEFAULT 0;
 DECLARE flagNOTFOUND BOOLEAN DEFAULT 0;
 DECLARE var1 VARCHAR(20);
 BEGIN
     DECLARE EXIT HANDLER FOR 1172
             SET flagPlusDun  :=1;
     DECLARE EXIT HANDLER FOR NOT FOUND
             SET flagNOTFOUND :=1;

     SELECT nom INTO var1 FROM bdsoutou.Pilote
            WHERE comp=p_comp;

    SELECT CONCAT('Le seul pilote de la compagnie ',
                  p_comp,' est ',var1)
        AS 'Resultat procException1';
 END;

 IF flagNOTFOUND THEN
  SELECT CONCAT('Il n''y a pas de pilote pour la
                compagnie ',p_comp)
       AS 'Resultat procException1';
 END IF;
 IF flagPlusDun THEN
  SELECT CONCAT('Il y a plusieurs pilotes pour la
                compagnie ',p_comp)
        AS 'Resultat procException1';
 END IF;
END;
```

Déclaration de la procédure et des variables.

Bloc qui déclare les deux exceptions.

Requête pouvant déclencher l'exception prévue.

Affichage du résultat.

Fin du bloc.

Gestion des erreurs.

Fin de la procédure.

4. Mise en place de déclencheurs (Triggers)

4.1. Qu'est ce qu'un déclencheur ?

Un déclencheur peut être vu comme des sous-programmes résidents associés à un événement particulier :

- une instruction INSERT, UPDATE, ou DELETE sur une table ou vue; (déclencheurs LMD)
- une instruction CREATE, ALTER, ou DROP sur un objet (table, vue, index, etc.); (déclencheurs LDD)
- le démarrage ou l'arrêt de la base, une erreur spécifique (not found, duplicate key, etc.), une connexion ou une déconnexion d'un utilisateur. On parle de déclencheurs d'instances.

À la différence des sous-programmes, l'exécution d'un déclencheur n'est pas explicite (par CALL par exemple), c'est l'événement lui même qui déclenche automatiquement le code programmé dans le déclencheur.

4.2. À quoi sert un déclencheur ?

En théorie, un déclencheur permet de :

- Programmer toutes les règles de gestion qui n'ont pas pu être mises en place par des contraintes au niveau des tables.
- Déporter des contraintes au niveau du serveur et alléger ainsi la programmation client.
- Renforcer des aspects de sécurité et d'audit.
- Programmer l'intégrité référentielle et la réplication dans des architectures distribuées, avec l'utilisation de liens de bases de données.

4.3. Déclencheur dans MySQL :

- les déclencheurs n'existent que depuis la version 5.
- Ils sont encore limitatifs en termes de fonctionnalités et relativement instables. En page d'accueil du site de MySQL, on peut lire dans MySQL 5.0 Triggers : « Triggers are very new. There are bugs. … Do not try triggers with a database that has important data in it… ».
- Prudence donc avec vos données. A partir de la version 5, il faudrait suivre de près toutes les limitations qui seront sans doute résolues au fur et à mesure des prochaines versions majeures du

serveur. Rappelons qu'avant la version 5.0.10, les déclencheurs ne pouvaient même pas accéder à la base !

- Mysql dans sa version 5, ne prend en charge que les déclencheurs de type LMD.

4.4. Syntaxe

```
CREATE TRIGGER nomDéclencheur
              { BEFORE | AFTER } { DELETE | INSERT | UPDATE }
       ON nomTable
       FOR EACH ROW
              {      instruction; |
                     [etiquette:] BEGIN
                                          instructions;
                                   END [etiquette];
              }
```

Note :

- Chaque enregistrement qui tente d'être supprimé d'une table, qui inclut un déclencheur de type DELETE FOR EACH ROW, est désigné par OLD au niveau du code du déclencheur. L'accès aux colonnes de ce pseudo-enregistrement dans le corps du déclencheur se fait par la notation pointée.
- Chaque enregistrement qui tente d'être ajouté dans une table est désigné par NEW au niveau du code du déclencheur. L'accès aux colonnes de ce pseudo-enregistrement dans le corps du déclencheur se fait par la notation pointée.

4.5. Invalidation dans le déclencheur

Une fonctionnalité importante des déclencheurs consiste à pouvoir invalider l'événement qui a déclenché l'action. En d'autres termes, c'est pouvoir faire dire au déclencheur non à une insertion, une modification ou à une suppression.

Dans tout déclencheur (de type BEFORE ou AFTER), une erreur lors de l'exécution et toutes les instructions du bloc sont invalidées.

L'invalidation dans un déclencheur se traduit en général par le déclenchement d'une exception (qui fait avorter l'instruction LMD), et par le retour d'un message d'erreur personnalisé.
Les procédures et déclencheurs MySQL ne permettent pour l'instant ni de provoquer une exception système ni de retourner un code SQL personnalisé. Il n'est pas non plus possible d'utiliser ROLLBACK dans un déclencheur

Une seule solution, qui n'est pas du tout satisfaisante, comme nous allons le voir, consisterait à provoquer artificiellement une erreur (mais pas une erreur système, par exemple accéder à une table inexistante). Il faut une erreur sémantiquement correcte qui pose problème à l'exécution (NULL dans une clé primaire).

Exemple :
Considérons la contrainte que « *tout pilote ne peut être qualifié sur plus de trois types d'appareils* ». Ici, il s'agit d'assurer la cohérence entre la valeur de la colonne nbQualif de la table Pilote et les enregistrements de la table Qualifications.

On crée d'abord la table trace :

```
CREATE TABLE Trace(col VARCHAR(80) PRIMARY KEY).
```

Ensuite:

```
CREATE TRIGGER TrigInsQualif
        BEFORE INSERT ON Qualifications
        FOR EACH ROW
BEGIN
        DECLARE v_compteur TINYINT(1);
        DECLARE v_nom VARCHAR(30);
        SELECT nbQualif, nom INTO v_compteur, v_nom
                FROM Pilote WHERE brevet = NEW.brevet;

        IF v_compteur < 3 THEN
                UPDATE Pilote
                SET nbQualif = nbQualif + 1
                WHERE brevet = NEW.brevet;
         ELSE
                INSERT INTO TRACE VALUES
                                (CONCAT('Le pilote ',v_nom, ' a déjà 3 qualifications!'));
                INSERT INTO TRACE VALUES (NULL);
        END IF;
END;

Et si on fait un ajout incorrect :
INSERT INTO Qualifications VALUES ('PL-1','A380', SYSDATE())$

ERROR 1048 (23000): Column 'col' cannot be null
-- ne fait pas l'INSERT dans Qualifications
-- ni dans Trace !
```

4.6. Suppression d'un déclencheur

Syntaxe :

```
DROP TRIGGER [nomBase.]nomDéclencheur;
```

Note : Le fait de détruire une table a pour conséquence d'effacer aussi tous les déclencheurs qui lui sont associés.

5. Utilisation du SQL dynamique

Une instruction SQL dynamique est stockée en tant que chaîne de caractères qui sera évaluée à l'exécution et non à la compilation (en opposition aux instructions SQL statiques qui peuplent la majorité des sous-programmes).

Commandes (Directives) associées :

1. Déclaration de l'instruction dynamique :

```
PREPARE nomEtat FROM étatPréparé;
```

- La commande PREPARE associe un nom à une instruction dynamique
- *étatPréparé* est une chaîne ou une variable de session contenant le texte de l'instruction SQL construite
- Dans cette chaîne, le caractère « ? » permet de se substituer à un paramètre

2. Exécution d'une instruction dynamique :

```
EXECUTE nomEtat [USING @var1 [, @var2] ...];
```

- L'instruction EXECUTE lance l'ordre paramétré avec éventuellement la clause USING qui reliera les paramètres aux variables de session.

3. Terminer une instruction dynamique :

```
DEALLOCATE PREPARE nomEtat
```

- supprime le contenu de l'ordre (une fin de session désalloue tous les ordres ouverts)

Exemple 1 : Considérons la table Avion suivante :

Immatriculation	TypeAv	NbHvol	Compagnie
F-GLFS	A320	1000	AH
F-WOWW	A380	1500	AH

La procédure suivante utilise la construction dynamique de l'extraction des avions dont le nombre d'heures de vol est égal à un paramètre spécifié par une variable de session (ici évaluée à 1 000). La requête est elle-même stockée dans une variable de session.

```
Delimiter $
-- Déclaration des variables de session
SET @vs_chaine = 'SELECT * FROM Avion WHERE nbHVol=?' $
SET @vs_nbhVol = 1000$

-- Déclaration de la procédure
CREATE PROCEDURE sousProg()
    BEGIN
            -- Préparation de l'ordre.
            PREPARE etat FROM @vs_chaine;

            -- Exécution de l'ordre
            EXECUTE etat USING @vs_nbhVol;

            DEALLOCATE PREPARE etat;
    END ;
    $

CALL sousProg()    -- Cet appel extrait l'avion immatriculé 'F-GLFS'.
```

---------- FIN ----------